

LangGraph Integration

Compress graph state, checkpoints, and long-term stores in LangGraph — smaller prompts, smaller checkpoint rows, lower token spend, with no changes to your graph's behavior.

1 Why use it

LangGraph state grows fast: RAG documents, agent scratchpads, accumulated tool outputs. Every superstep rewrites the whole checkpoint row, every node forwards bloated context into the next prompt. Compress shrinks the largest text fields at three independent layers — pick any combination:

- **Graph nodes** — drop a compression step into your `StateGraph`.
- **Checkpoint serializer** — compress strings *before* they hit Postgres/Redis. Smaller rows, less write amplification.
- **Long-term store** — compress values on write into any `BaseStore`.

Fail-open by default: if the Compress API is unreachable, the original text flows through unchanged and your graph keeps running.

2 Install

```
pip install 'compress[langgraph]'
export COMPRESR_API_KEY=cmp...
```

3 Compression as a graph node

Drop a Compress node between your retriever and your LLM call. The node reads a string field from state, compresses it against the user's query, and writes it back.

```
import os
from langgraph.graph import StateGraph, START, END
from compress.integrations.langgraph import make_compress_node

graph = StateGraph(dict)

graph.add_node("retrieve", retrieve_docs)           # your retriever
graph.add_node(
    "compress",
    make_compress_node(
        api_key=os.environ["COMPRESR_API_KEY"],
        context_key="retrieved_text",              # field to compress
        query_key="user_question",                 # query field
        target_compression_ratio=0.5,               # remove ~50% of tokens
        min_tokens=200,                             # skip if shorter
    ),
)
graph.add_node("answer", call_llm)

graph.add_edge(START, "retrieve")
graph.add_edge("retrieve", "compress")
graph.add_edge("compress", "answer")
graph.add_edge("answer", END)

app = graph.compile()
```

4 Compressed checkpoints

Wrap your checkpoint saver's serializer to compress long strings *before* they get persisted. Smaller checkpoint rows mean faster writes, less storage cost, and less TOAST write amplification on Postgres.

```
import os
from langgraph.checkpoint.postgres import PostgresSaver
from compress.integrations.langgraph import CompressCheckpointSerializer

saver = PostgresSaver(
    connection_string="postgresql://...",
    serde=CompressCheckpointSerializer(
        api_key=os.environ["COMPRESR_API_KEY"],
        min_tokens=500,                       # gate by size: skip short strings
        target_compression_ratio=0.6,
    ),
)
```

The checkpoint serializer gates by size with `min_tokens`: checkpointers serialize each channel value on its own, so the serializer never sees the field name and the `fields` allowlist does not apply here — use `fields` on `CompressStore` (next section), which receives the whole value dict. Keep large paraphrasable channels (RAG dumps, scratchpads) above the threshold and short verbatim channels (IDs, status codes) naturally fall below it.

Note

Lossy by design. Checkpoint and store compression is one-way: reads return the compressed string, not the original. Scope it to paraphrasable payloads (RAG dumps, scratchpads, transcripts) — keyed by `min_tokens` for checkpoints and by `fields` for the store — and keep anything you need verbatim (IDs, status codes, structured outputs) out of the compressed path.

5 Compressed long-term store

Wrap any LangGraph `BaseStore` (`InMemoryStore`, `PostgresStore`, `RedisStore`). String fields above the threshold are compressed on `put`; reads pass straight through.

```
import os
from langgraph.store.memory import InMemoryStore
from compresr.integrations.langgraph import CompresrStore

inner = InMemoryStore()
store = CompresrStore(
    inner,
    api_key=os.environ["COMPRESR_API_KEY"],
    fields={"retrieved_text", "scratchpad"},
    min_tokens=500,
    target_compression_ratio=0.6,
)

store.put(("user_42",), "doc_1", {
    "retrieved_text": "<long RAG dump>",      # compressed
    "title": "Order #4711 case notes",        # untouched
})

item = store.get(("user_42",), "doc_1")      # reads pass through
```

A fourth helper, `compresr_handoff_tool(agent_name, ...)`, builds a supervisor handoff tool that compresses `task_description` and `context` in flight:

```
from compresr.integrations.langgraph import compresr_handoff_tool

handoff = compresr_handoff_tool(
    "researcher",                                # target agent name
    api_key=os.environ["COMPRESR_API_KEY"],
    target_compression_ratio=0.6,
)
```

For `langchain.agents.create_agent` pipelines, the same package also exposes `CompresrToolMiddleware`, `CompresrSummarizationMiddleware` and `CompresrPromptMiddleware` — see the LangChain integration PDF for usage.

6 Key parameters

Parameter	Default	Description
<code>api_key</code>	<code>None</code>	Your Compresr API key. Required unless you pass a pre-built <code>client</code> .
<code>client</code>	<code>None</code>	Share a single <code>CompressionClient</code> across components for pooled transport, custom timeouts, or proxy/mTLS setups.
<code>compression_model</code>	<code>latte_v1</code>	<code>latte_v1</code> is the default query-aware model; <code>latte_v2</code> is the faster variant.
<code>target_compression_ratio</code>	<code>0.5</code>	Strength. Values in <code>[0,1]</code> remove that fraction of tokens (<code>0.5 = ~50%</code> smaller). Values <code>> 1</code> apply an <code>Nx</code> factor (<code>4 = ~4x</code> smaller).
<code>min_tokens</code>	<code>200 (node) / 500 (store, checkpoint)</code>	Skip compression below this token count — keeps short strings cheap.
<code>fields</code>	<code>None</code>	(<code>CompresrStore</code> only.) Allowlist of dict keys eligible for compression. Not applied by the checkpoint serializer, which gates by <code>min_tokens</code> (it sees one channel value at a time, without its key).
<code>context_key</code>	—	(Node only, required.) State field to compress.
<code>query_key</code>	<code>None</code>	(Node only.) State field used as the compression query (e.g. <code>"user_question"</code>).
<code>on_error</code>	<code>"passthrough"</code>	<code>"passthrough"</code> = fail-open, log and return the original text. <code>"raise"</code> = propagate the exception.